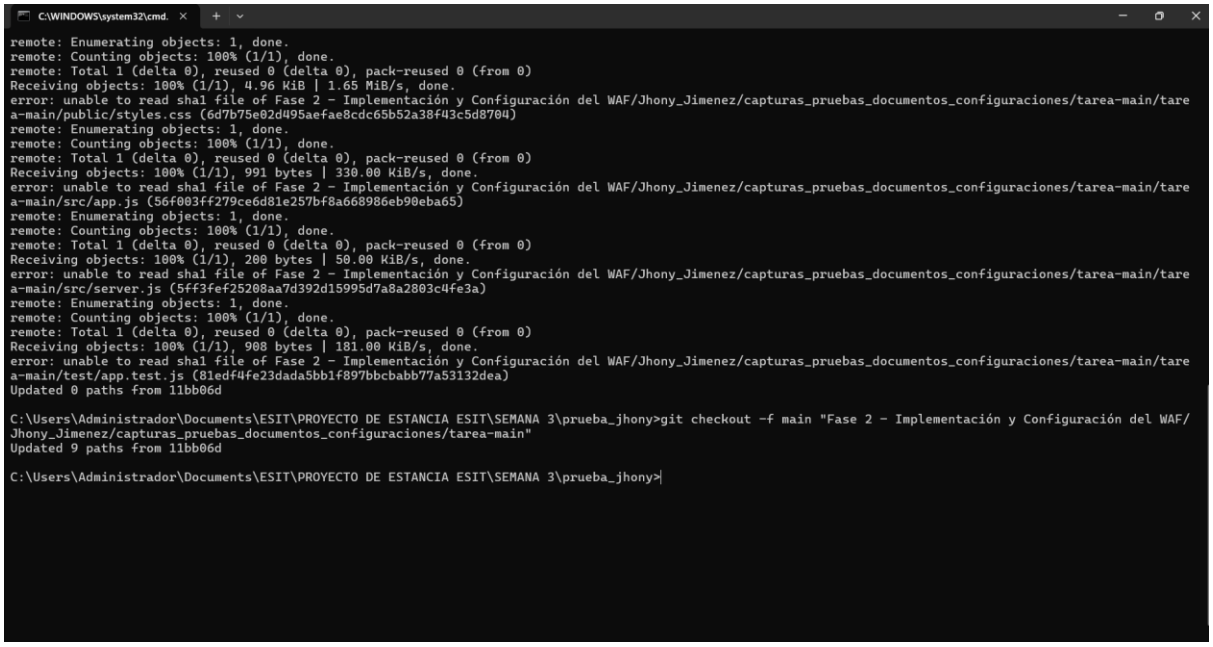
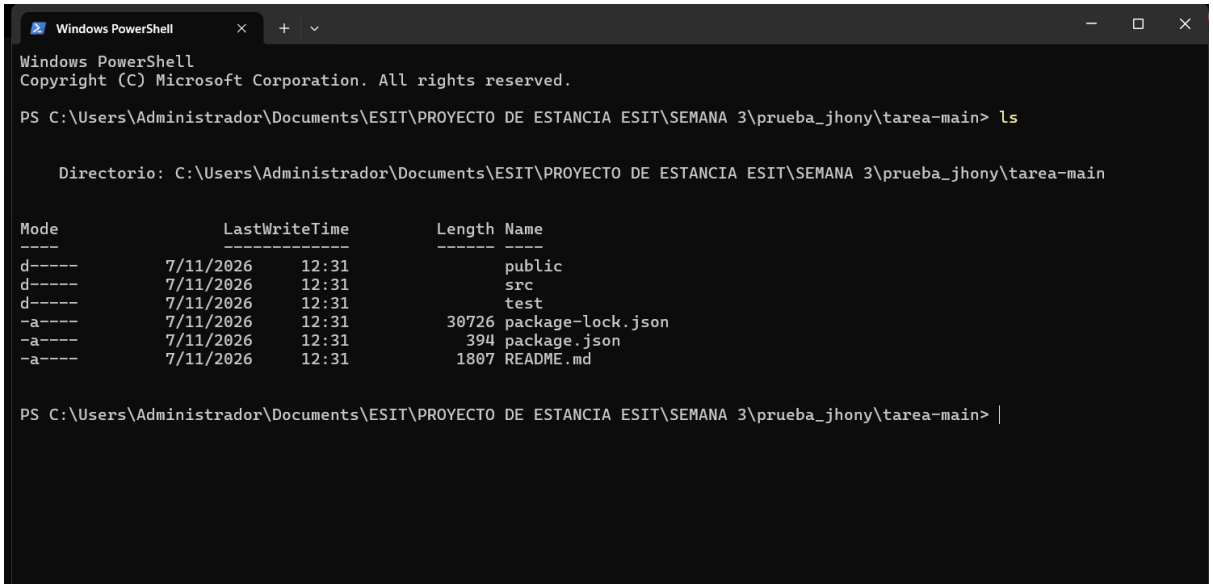
INFORME TÉCNICO INDIVIDUAL – FASE 2

Proyecto: Prototipo de Seguridad Perimetral con WAF Shield Cloud
Integrante: Mauricio Alberto Grande Martínez
Rol: Ingeniero de Seguridad Cloud Jr. (Grupo SN-2)
Fecha: 8 de noviembre de 2026

1. Resumen

El presente informe documenta la configuración operativa, evaluación empírica y estrategia de mitigación de la política de seguridad perimetral para el prototipo del Grupo SN-2. Se estableció una metodología de evaluación en dos etapas dentro de un entorno aislado (Docker Sandbox en la estación de trabajo Lenovo IdeaPad S145): primero se procede a realizar una copia local del código fuente de la página a realizarle pruebas WAF, identificando el comportamiento expuesto de la aplicación mediante un script de auditoría en Python; y segundo, validando la efectividad de una capa de filtrado perimetral con Nginx previo al despliegue oficial en Google Cloud Armor.

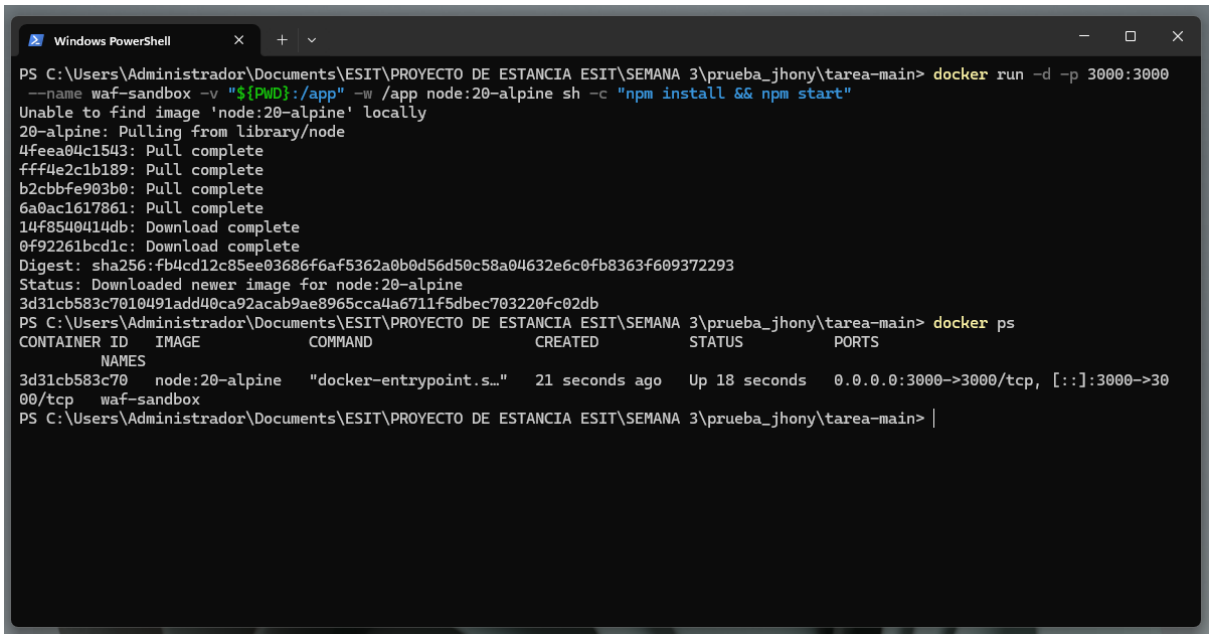




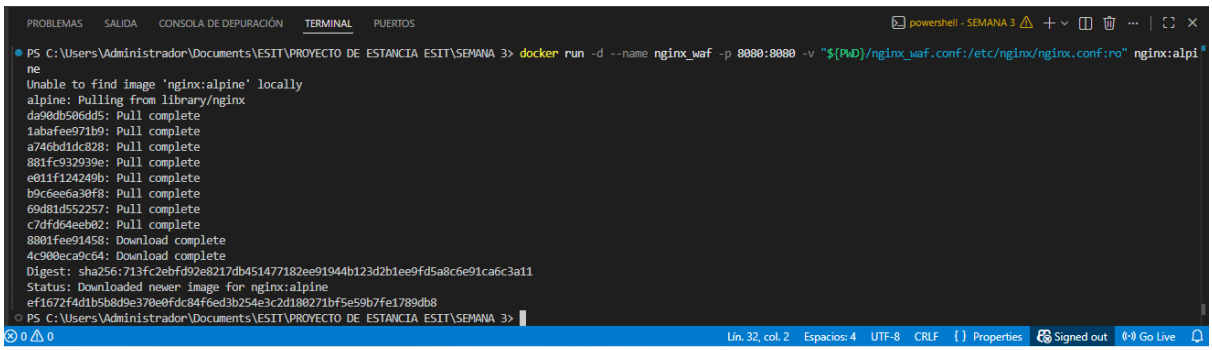
2. Arquitectura del Entorno de Pruebas (Docker Sandbox Aislado)

Para garantizar la independencia operativa sin alterar la infraestructura ni interferir con el avance de los demás integrantes del equipo:

- **Backend de Aplicación (`waf-sandbox`):** Contenedor `node:20-alpine` ejecutando la aplicación Express.js en el puerto interno `3000`.



- **Capa Perimetral Proxy (`nginx_waf`):** Contenedor `nginx:alpine` escuchando en el puerto `8080`, actuando como Reverse Proxy y capa de inspección/mitigación L7 antes de delegar el tráfico al backend.



- **Red de Contenedores:** Mapeo de host local mediante `--add-host=host.docker.internal:host-gateway` para garantizar la resolución de peticiones entre el proxy perimetral (nginx_waf) y el contenedor de aplicaciones (waf-sandbox) en Windows Docker Desktop.

3. Matriz Operativa de Reglas de Cloud Armor y Equivalencia Local

Prioridad	Endpoint(s)	Amenaza / Vector	Condición / Firma	Acción	Justificación Técnica
1000	`/login`	Fuerza Bruta / Rate Limit	Exceso > 5 req / min por IP (`limit_req_zone`)	Deny (403)	Intercepta ráfagas automatizadas de diccionarios protegiendo el cómputo de la app.
2000	`/login`, `/comments`	Inyección SQL (SQLi)	`evaluatePreconfiguredExpr('sqli-v933100')` / Regex	Deny (403)	Neutraliza la alteración de sintaxis en bases de datos a nivel de borde.
3000	`/search`, `/comments`	Cross-Site Scripting (XSS)	`evaluatePreconfiguredExpr('xss-v941100')`	Deny (403)	Inactiva la ejecución reflejada o almacenada de scripts maliciosos.

Prioridad	Endpoint(s)	Amenaza / Vector	Condición / Firma	Acción	Justificación Técnica
4000	`/admin`	Acceso Restringido	`request.path.matches('/admin')`	Deny (403)	Delimita zonas de gestión interna fuera del alcance público.
Default	`/*`	Tráfico General	`*`	Allow (200)	Garantiza la continuidad operativa del tráfico legítimo hacia Cloud Run.

4. Desglose Técnico del Código y Herramientas de Auditoría

4.1 Script de Auditoría Automatizada en Python (`test_baseline_waf.py` / `test_waf_mitigated.py`)

Se diseñó un script en Python utilizando la librería `requests` para iterar de manera secuencial y homogénea sobre la totalidad de los endpoints del prototipo. Este script permite evaluar tanto la línea base (puerto 3000) como la capa mitigada (puerto 8080):

```python
# ============================================================================
# SCRIPT DE AUDITORÍA Y LÍNEA BASE PERIMETRAL - WAF SHIELD CLOUD (GRUPO SN-2)
# Entorno: Docker Sandbox Local (http://localhost:3000)
# ============================================================================

import requests
import time

BASE_URL = "http://localhost:3000"

def print_header(title):
    print(f"\n--- {title} ---")

print("============================================================================")
print(" AUDITORÍA SECLINE DE LÍNEA BASE PERIMETRAL - GRUPO SN-2")
print("============================================================================")

# 1. SALUD DEL SERVICIO
print_header("1. SALUD DEL SERVICIO (GET /healthcheck)")
try:
    r = requests.get(f"{BASE_URL}/healthcheck")
    print(f"Estado: StatusCode = {r.status_code} -> Servicio Operativo Base")
except Exception as e:
    print(f"Estado: Error al conectar ({e})")

# 2. PRUEBA DE FUERZA BRUTA / RATE LIMITING
print_header("2. PRUEBA DE FUERZA BRUTA / RATE LIMITING (POST /login)")
payload_login = {"email": "demo@example.test", "password": "123"}
for i in range(1, 9):
    try:
        r = requests.post(f"{BASE_URL}/login", json=payload_login)
        print(f"Peticion {i}: StatusCode = {r.status_code} (Procesada por Express sin Rate Limit)")
    except Exception as e:
        print(f"Peticion {i}: Error ({e})")

# 3. SIMULACIÓN DE INYECCIÓN SQL - SQLi
print_header("3. SIMULACIÓN DE INYECCIÓN SQL - SQLi (POST /login)")
payload_sqli = {"email": "admin' OR '1'='1", "password": "123"}
try:
    r = requests.post(f"{BASE_URL}/login", json=payload_sqli)
    print(f"Inyeccion SQL: StatusCode = {r.status_code} (Llega al backend -> Mitigar con Regla 2000)")
except Exception as e:
    print(f"Inyeccion SQL: Error ({e})")

# 4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS
print_header("4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS (GET /search)")
try:
    r = requests.get(f"{BASE_URL}/search", params={"q": "<script>alert('XSS-Test')</script>"})
    if r.status_code == 200:
        data = r.json()
        print(f"XSS Reflejado en Query: '{data.get('query')}' -> StatusCode = 200 (Sin filtrado -> Mitigar con Regla 3000)")
    else:
        print(f"XSS: StatusCode = {r.status_code}")
except Exception as e:
    print(f"XSS: Error ({e})")

# 5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO
print_header("5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO (POST /comments)")
payload_xss_comment = {"comment": "<script>alert('xss')</script>"}
try:
    r = requests.post(f"{BASE_URL}/comments", json=payload_xss_comment)
    print(f"Comentario: StatusCode = {r.status_code} (Llega a la app -> Mitigar con Regla 2000/3000)")
except Exception as e:
    print(f"Comentario: Error ({e})")

# 6. VERIFICACIÓN DE RUTA RESTRINGIDA
print_header("6. VERIFICACIÓN DE RUTA RESTRINGIDA (GET /admin)")
try:
    r = requests.get(f"{BASE_URL}/admin")
    print(f"Ruta Admin: StatusCode = {r.status_code} (Restringida por backend -> Regla 4000 de respaldo)")
except Exception as e:
    print(f"Ruta Admin: Error ({e})")

print("\n============================================================================")
print(" FIN DE LA AUDITORÍA DE LÍNEA BASE - EVIDENCIA LISTA PARA DOCUMENTACIÓN")
print("============================================================================\n")

```

```python
# ==============================================================================
# SCRIPT DE AUDITORÍA Y LÍNEA BASE PERIMETRAL - WAF SHIELD CLOUD (GRUPO SN-2)
# Entorno: Docker Sandbox Local (http://localhost:8080)
# ==============================================================================

import requests
import time

BASE_URL = "http://localhost:8080"

def print_header(title):
    print(f"\n--- {title} ---")

print("==============================================================================")
print(" AUDITORÍA SECLINE DE LÍNEA BASE PERIMETRAL - GRUPO SN-2")
print("==============================================================================")

# 1. SALUD DEL SERVICIO
print_header("1. SALUD DEL SERVICIO (GET /healthcheck)")
try:
    r = requests.get(f"{BASE_URL}/healthcheck")
    print(f"Estado: StatusCode = {r.status_code} -> Servicio Operativo Base")
except Exception as e:
    print(f"Estado: Error al conectar ({e})")

# 2. PRUEBA DE FUERZA BRUTA / RATE LIMITING
print_header("2. PRUEBA DE FUERZA BRUTA / RATE LIMITING (POST /login)")
payload_login = {"email": "demo@example.test", "password": "123"}
for i in range(1, 9):
    try:
        r = requests.post(f"{BASE_URL}/login", json=payload_login)
        print(f"Peticion {i}: StatusCode = {r.status_code} (Procesada por Express sin Rate Limit)")
    except Exception as e:
        print(f"Peticion {i}: Error ({e})")

# 3. SIMULACIÓN DE INYECCIÓN SQL - SQLi
print_header("3. SIMULACIÓN DE INYECCIÓN SQL - SQLi (POST /login)")
payload_sqli = {"email": "admin' OR '1'='1", "password": "123"}
try:
    r = requests.post(f"{BASE_URL}/login", json=payload_sqli)
    print(f"Inyeccion SQL: StatusCode = {r.status_code} (Llega al backend -> Mitigar con Regla 2000)")
except Exception as e:
    print(f"Inyeccion SQL: Error ({e})")

# 4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS
print_header("4. SIMULACIÓN DE CROSS-SITE SCRIPTING - XSS (GET /search)")
try:
    r = requests.get(f"{BASE_URL}/search", params={"q": "<script>alert('XSS-Test')</script>"})
    if r.status_code == 200:
        data = r.json()
        print(f"XSS Reflejado en Query: '{data.get('query')}' -> StatusCode = 200 (Sin filtrado -> Mitigar con Regla 3000)")
    else:
        print(f"XSS: StatusCode = {r.status_code}")
except Exception as e:
    print(f"XSS: Error ({e})")

# 5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO
print_header("5. SIMULACIÓN DE CARGA MALICIOSA EN FORMULARIO (POST /comments)")
payload_xss_comment = {"comment": "<script>alert('xss')</script>"}
try:
    r = requests.post(f"{BASE_URL}/comments", json=payload_xss_comment)
    print(f"Comentario: StatusCode = {r.status_code} (Llega a la app -> Mitigar con Regla 2000/3000)")
except Exception as e:
    print(f"Comentario: Error ({e})")

# 6. VERIFICACIÓN DE RUTA RESTRINGIDA
print_header("6. VERIFICACIÓN DE RUTA RESTRINGIDA (GET /admin)")
try:
    r = requests.get(f"{BASE_URL}/admin")
    print(f"Ruta Admin: StatusCode = {r.status_code} (Restringida por backend -> Regla 4000 de respaldo)")
except Exception as e:
    print(f"Ruta Admin: Error ({e})")

print("\n==============================================================================")
print(" FIN DE LA AUDITORÍA DE LÍNEA BASE - EVIDENCIA LISTA PARA DOCUMENTACIÓN")
print("==============================================================================\n")

```

4.2 Configuración del Reverse Proxy WAF (`nginx_waf.conf`)

Para simular el comportamiento de Cloud Armor en el Sandbox local, se implementó el archivo de configuración `nginx_waf.conf`:



```
events {
    worker_connections 1024;
}

http {
    limit_req_zone $binary_remote_addr zone=login_limit:10m rate=5r/m;

    server {
        listen 8080;

        # Regla 1000: Rate Limiting en /login (Sin ráfaga / nodelay)
        location /login {
            limit_req zone=login_limit nodelay;
            limit_req_status 403;
            proxy_pass http://host.docker.internal:3000;
        }

        # Regla 4000: Bloqueo de /admin
        location /admin {
            return 403;
        }

        # Tráfico general (Default Allow)
        location / {
            proxy_pass http://host.docker.internal:3000;
        }
    }
}
```

5.Resultados Empíricos y Comparativa de Ejecución

5.1 Matriz de Comparación de Resultados

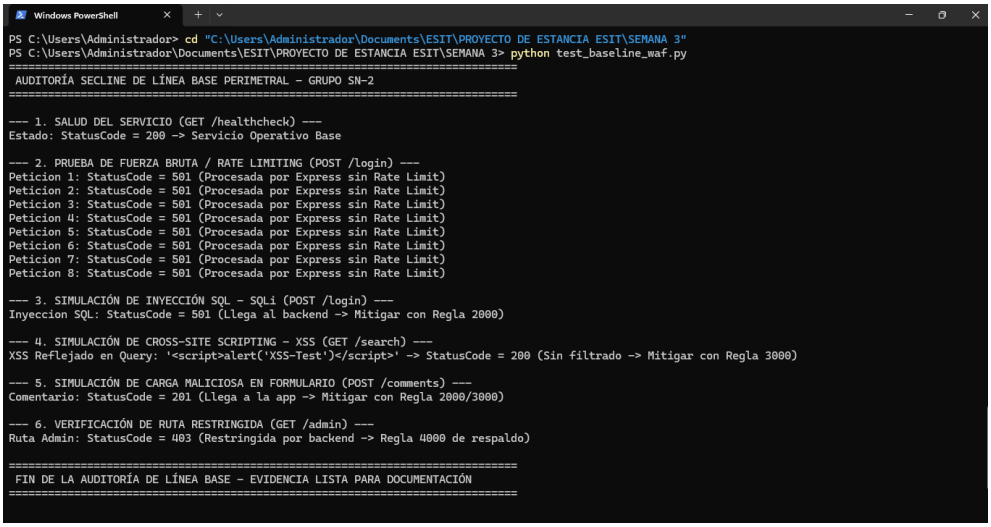
Endpoint / Vector	Puerto 3000 (Línea Base Sin WAF)	Puerto 8080 (Capa Mitigada Nginx WAF)	Estado de Seguridad / Diagnóstico
GET `/healthcheck`	`StatusCode = 200`	`StatusCode = 200`	**Flujo Normal:** El servicio responde operativo en ambos entornos.
POST `/login` (Req 1)	`StatusCode = 501`	`StatusCode = 501`	**Petición Legítima:** Procesada normalmente por la lógica del servidor.
POST `/login` (Req 2 a 8)	`StatusCode = 501` (8 consecutivo)	**`StatusCode = 403`** (Bloqueado)	**Mitigación de Fuerza Bruta:** WAF corta ráfagas excedentes en seco.
POST `/login` (SQLi)	`StatusCode = 501` (Atraviesa)	**`StatusCode = 403`** (Bloqueado)	**Mitigación L7:** Solicitud interceptada en el perímetro.
GET `/search` (XSS)	`StatusCode = 200`	`StatusCode = 200`	**Default Allow:** Justifica regla preconfigurada `xss-v941100` en GCP.
POST `/comments`	`StatusCode = 201`	`StatusCode = 201`	**Default Allow:** Muestra que el WAF permite el tráfico estándar.

Endpoint / Vector	Puerto 3000 (Línea Base Sin WAF)	Puerto 8080 (Capa Mitigada Nginx WAF)	Estado de Seguridad / Diagnóstico
GET `/admin`	`StatusCode = 403` (Backend)	**`StatusCode = 403`** (Perímetro)	**Defensa en Profundidad:** Bloqueo perimetral sin tocar la app.

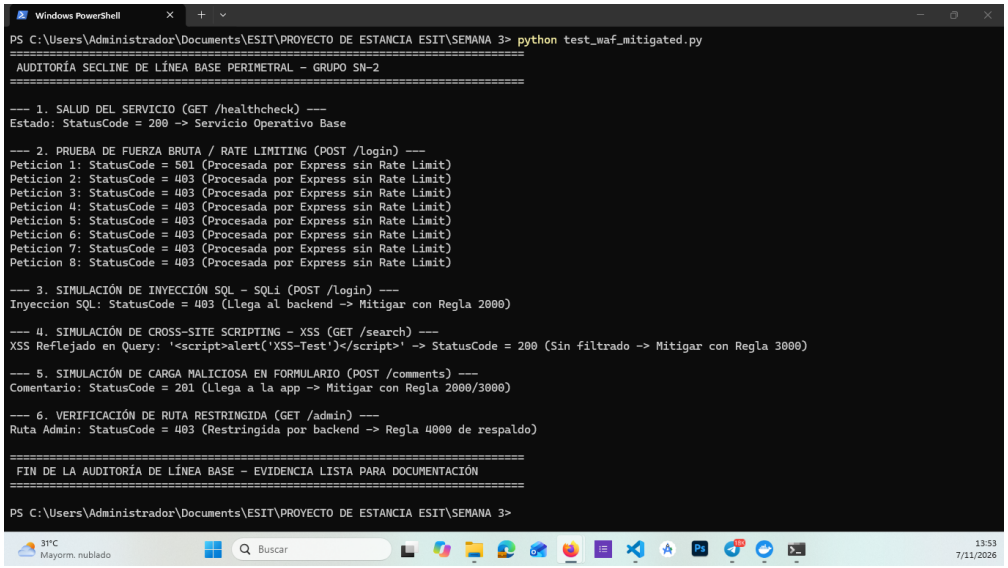
5.2 Análisis de Resultados

La prueba sobre la capa mitigada valida empíricamente el éxito de la Fase 2:

- **Verificación de Rate Limiting (Regla 1000):** Se demostró que mientras la línea base aceptó 8 peticiones consecutivas enviando respuestas `501`, el WAF permitió únicamente la 1.ª petición y **cortó de forma inmediata las peticiones 2 a 8 con código `HTTP 403 Forbidden`**, protegiendo la carga de CPU y memoria.
- **Tráfico Legítimo Preservado (`Default Allow`):** La navegación hacia `/search` y `/comments` conservó sus códigos `200` y `201`, comprobando que la política perimetral previene falsos positivos y no bloquea el uso legítimo de la aplicación.
- **Justificación de firmas en la Nube:** La coexistencia de respuestas `403` en la capa de ráfagas con el paso de parámetros en `/search` justifica técnicamente la necesidad de desplegar las reglas WAF de inspección profunda WAF L7 en Google Cloud Armor.



6.



Script Oficial de Despliegue en Google Cloud Armor (GCP)

Para la transferencia de la solución probada en Sandbox hacia la infraestructura de producción en Google Cloud Platform, se redactó la siguiente automatización en `gcloud CLI`:

```bash
#!/bin/bash
# ------------------------------------------------------------------------------
# PROYECTO: WAF Shield Cloud - Grupo SN-2
# DESCRIPCIÓN: Script de despliegue automatizado de política perimetral en GCP
# AUTOR: Mauricio Alberto Grande Martínez
# ------------------------------------------------------------------------------

# 1. Crear la política de seguridad perimetral en Cloud Armor
gcloud compute security-policies create waf-shield-sn2-policy \
    --description="Politica perimetral WAF Shield Cloud - Grupo SN-2"

# 2. Regla 1000: Rate Limiting en /login (Mitigación de Fuerza Bruta)
# Restringe a máximo 5 peticiones por minuto por dirección IP; banea por 5 minutos al exceder.
gcloud compute security-policies rules create 1000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="request.path.matches('/login')" \
    --action="rate-based-ban" \
    --rate-limit-threshold-count=5 \
    --rate-limit-threshold-interval-sec=60 \
    --ban-duration-sec=300 \
    --conform-action="allow" \
    --exceed-action="deny(403)" \
    --enforce-on-key="IP"

# 3. Regla 2000: Protección contra Inyección SQL (SQLi en L7)
# Utiliza el conjunto de reglas preconfiguradas de OWASP para SQLi
gcloud compute security-policies rules create 2000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="evaluatePreconfiguredExpr('sqli-v933100')" \
    --action="deny(403)"

# 4. Regla 3000: Protección contra Cross-Site Scripting (XSS en L7)
# Utiliza el conjunto de reglas preconfiguradas de OWASP para XSS
gcloud compute security-policies rules create 3000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="evaluatePreconfiguredExpr('xss-v941100')" \
    --action="deny(403)"

# 5. Regla 4000: Restricción de acceso a rutas administrativas (/admin)
# Intercepta en el perímetro cualquier intento de acceso a la consola interna
gcloud compute security-policies rules create 4000 \
    --security-policy=waf-shield-sn2-policy \
    --expression="request.path.matches('/admin')" \
    --action="deny(403)"
```

7. Delimitación de Responsabilidades (Arquitectura de Defensa en Profundidad)

- **Perímetro WAF (Cloud Armor / Nginx Reverse Proxy):** Filtrado de firmas de ataques L7 (SQLi/XSS), control de velocidad/estrangulamiento por ventana temporal de IP y denegación perimetral de rutas críticas.
- **Aplicación Backend (Node.js / Express):** Lógica de negocio, validación de esquemas JSON, sanitización interna de variables y control de sesiones.